

AceCampus Learning Roadmap

May 28, 2026

Contents

1	Phase 1: Core Languages (The Foundation)	2
2	Phase 2: Web Development & APIs	3
3	Phase 3: Entering the AI Space (AI & LLM)	4
4	Phase 4: Deployment & Operations (Cloud & DevOps)	5
5	Phase 5: Making AI Production-Ready (AI Evaluation & Ops)	7
6	Phase 6: Expanding the Arsenal (Good to Have)	8

How to Use This Roadmap

This roadmap is ordered by dependency and impact. Each phase builds on the previous one. Master the topics in a phase before moving to the next.

1 Phase 1: Core Languages (The Foundation)

Start here. This phase sets up the core logic and data handling for everything you will build.

Python (Compulsory)

Master this first. It is the absolute backbone of the modern tech stack, especially for building machine learning models and AI-integrated platforms.

- Fundamentals: syntax, variables, types, operators, and expressions
- Control flow: conditionals, loops, comprehensions
- Data structures: lists, tuples, sets, dictionaries
- Functions: parameters, scope, recursion, decorators
- OOP: classes, inheritance, polymorphism, dunder methods
- Modules and packages: imports, virtual environments, pip
- Error handling: exceptions, custom errors, logging
- File I/O: text, CSV, JSON, filesystem paths
- Standard library essentials: datetime, pathlib, itertools, collections
- Type hints and basic typing: typing module, mypy concepts
- Testing: unittest, pytest basics
- Algorithmic thinking: complexity, common patterns

Java (Chosen over .NET)

Focus on Java next. It is highly versatile, has a massive beginner community, and aligns perfectly with standard engineering curriculums.

- JDK setup and tooling: javac, JVM, JRE
- Syntax and types: primitives, references, casting
- OOP: classes, interfaces, abstract classes
- Collections: List, Set, Map, generics
- Exceptions: checked vs unchecked, custom exceptions
- Lambdas and streams: functional style, stream operations
- Concurrency basics: threads, synchronization
- Build tools: Maven or Gradle basics
- Testing: JUnit, test structure

SQL

Learn how to write queries to store, retrieve, and manage data in relational databases.

- Relational model: tables, rows, columns, keys
- CRUD queries: SELECT, INSERT, UPDATE, DELETE
- Filtering and sorting: WHERE, ORDER BY, LIMIT
- Joins: INNER, LEFT, RIGHT, FULL
- Aggregation: GROUP BY, HAVING, COUNT, SUM
- Subqueries and CTEs
- Indexes and query performance basics
- Normalization and constraints
- Transactions and isolation levels

2 Phase 2: Web Development & APIs

Next, learn how to build user interfaces and connect them to your backend logic.

JavaScript (Chosen over TypeScript)

Start with vanilla JS to understand web behavior before dealing with strict typing rules.

- Language basics: variables, functions, scope, closures
- DOM manipulation: selectors, events, updates
- Asynchronous JS: callbacks, promises, async/await
- Networking: fetch, REST calls, JSON handling
- Browser APIs: localStorage, sessionStorage
- Modules and bundling concepts
- Security basics: CORS, XSS, CSRF awareness

React (Chosen over Angular)

React has a gentler learning curve, a component-based structure, and is the industry standard for modern web platforms.

- JSX and component composition
- Props and state
- Hooks: useState, useEffect, useMemo, useCallback
- Forms and validation
- Routing basics
- State management: Context, lifting state up

- Data fetching patterns
- Testing: React Testing Library basics
- Performance: memoization, lazy loading

REST APIs

Learn how to create APIs that allow your React frontend to communicate with your Python or Java backend.

- HTTP methods and status codes
- Resource design and naming
- Request and response schemas
- Validation and error handling
- Authentication and authorization: JWT, OAuth basics
- Pagination, filtering, sorting
- Versioning strategies
- Documentation: OpenAPI and Swagger
- Rate limiting and basic security

3 Phase 3: Entering the AI Space (AI & LLM)

With full-stack skills secured, you can start integrating AI. Use AI coding assistants (like GitHub Copilot or Claude) while learning this phase to accelerate your progress.

Prompt Engineering & Structured Outputs

Learn how to craft prompts that force LLMs to return consistent, usable data (like JSON) for your applications.

- Prompt structure: system, developer, user roles
- Few-shot and example-driven prompts
- Instruction clarity and constraint phrasing
- JSON schemas and output validation
- Tool or function calling basics
- Prompt templates and versioning
- Safety and refusal handling

LLMs & Embeddings

Understand how Large Language Models work and how text is converted into vector numbers (embeddings).

- Tokens, context windows, and model limits
- Embeddings and similarity metrics
- Chunking strategies and preprocessing
- Model selection tradeoffs: cost vs quality
- Fine-tuning vs prompt engineering basics

RAG Pipelines

Learn Retrieval-Augmented Generation to allow LLMs to answer questions using your own custom data and documents.

- Data ingestion and cleaning
- Chunking and embedding creation
- Vector storage and metadata filters
- Retrieval strategies: dense, sparse, hybrid
- Reranking and context selection
- Grounded answers and citations
- Caching and data refresh policies

LangChain (Chosen over LangGraph, CrewAI, and AutoGen)

The most accessible framework for orchestrating LLMs and chaining commands together.

- Core abstractions: prompts, chains, tools
- Memory and conversational state
- Retrievers and vector store integrations
- Agents and tool routing
- LCEL basics and composability
- Tracing and debugging workflows

4 Phase 4: Deployment & Operations (Cloud & DevOps)

Take your local application and put it on the internet automatically.

Git

Essential for version control and code management.

- Repositories, commits, and history
- Branching and merging
- Pull requests and code reviews
- Tags and releases
- .gitignore and repo hygiene

Containers

Learn Docker to package your apps so they run perfectly on any machine.

- Dockerfile basics
- Images, containers, and registries
- Volumes and data persistence
- Networking and ports
- Docker Compose for multi-service apps

AWS (Chosen over Azure and GCP)

AWS is the undisputed market leader with the most beginner tutorials available.

- IAM: users, roles, policies
- Compute: EC2, Lambda, ECS
- Storage: S3
- Databases: RDS, DynamoDB basics
- Networking: VPC, security groups
- Monitoring: CloudWatch
- Container registry: ECR

CI/CD Pipelines

Automate your testing and deployment processes so your code updates live seamlessly.

- Build and test stages
- Linting and quality gates
- Artifact creation and storage
- Secrets management
- Deployment strategies: blue/green, canary
- Rollbacks and release management

5 Phase 5: Making AI Production-Ready (AI Evaluation & Ops)

Ensure your deployed AI apps are stable, safe, and cost-effective.

AI Quality Metrics

Measure the accuracy and relevance of your AI's outputs.

- Groundedness and faithfulness
- Answer relevance and completeness
- Hallucination rate tracking
- A/B testing and regression suites
- Human evaluation workflows

Monitoring & Guardrails

Prevent the AI from going off-topic or returning harmful content.

- Input and output validation
- Prompt injection defenses
- Safety filters and policy checks
- Observability: logs, traces, metrics
- Alerting and incident response

Cost & Latency Optimisation

Learn how to reduce the computing power, time, and money required for each AI request.

- Model selection and tiering
- Caching and reuse of results
- Context trimming and summarization
- Batching and streaming responses
- Token budgeting and limits

Fallback & Degradation Strategies

Ensure your platform still functions (even with limited features) if the AI API crashes.

- Retries and circuit breakers
- Graceful degradation paths
- Rule-based or template fallbacks
- Cached responses and offline modes
- SLA-aware routing

6 Phase 6: Expanding the Arsenal (Good to Have)

Once the core pipeline is mastered, add these tools to specialize your skillset.

Streamlit (Chosen over FastAPI)

Build highly interactive web interfaces for your data and AI scripts using pure Python.

- App layout and components
- Widgets and user input
- Session state and caching
- File upload and data display
- Multi-page apps and deployment

Vector DB Concepts

Learn how to use databases designed specifically for storing and searching AI embeddings.

- Vector indexing (HNSW and IVF basics)
- Similarity search and filtering
- Metadata schemas and hybrid search
- Consistency and update strategies
- Evaluation of recall and latency

RAGAS (Chosen over DeepEval)

A straightforward tool specifically built to evaluate how well your RAG pipelines are performing.

- Metric selection and setup
- Ground truth dataset creation
- Faithfulness and relevance scoring
- Automated evaluation workflows

Event-Driven Architectures

Build systems that react instantly to specific triggers or events in your application.

- Events, queues, and pub/sub
- Webhooks and event routing
- Idempotency and retries
- Event schema design and versioning
- Monitoring and dead-letter queues

Kubernetes

Save this for the very end. It is complex but powerful for managing massive, scaled container deployments.

- Pods, deployments, and services
- ConfigMaps and secrets
- Ingress and networking basics
- Auto-scaling and resource limits
- Helm and basic cluster operations